PGR Platform

Functional Make-It-Real Plan

Every feature that **looks** implemented in the UI but relies on a workaround, stub, illustrative data, or a missing flow underneath. Deployment / infra is out of scope here — see the earlier plan for that.

Each gap is broken into a fullstack build: data model · service · API · UI · tests.

v1 · 2026-08-24

Severity	Meaning (user impact, not infra)
HIGH	The screen makes a promise the platform cannot keep — administrators will discover the shortfall on their first real
MED	The flow is thin — usable, but forces the office back onto email / Excel for what looks like a first-class feature.
LOW	Cosmetic or convenience — real, but noticeably below the polish of everything around it.

Functional roadmap (6 blocks, ordered by user harm)

Block	Weeks	Theme	What gets real
F1	1–2	Statutory truth	HESA profile signed off; report profile audit trail
F2	3–4	Person integrity	Person merge · contacts · GDPR export/erase · dedupe UI
F3	5–6	Recruitment depth	Route badge/filter · references · interviews · conditional offers · fee-status/visa
F4	7–8	Award pipeline	Corrections upload flow · classification workflow · certificate PDF
F5	9–10	Ops truth	Workflow SLA timers · reconciliation remediation · funding project backfill
F6	11–12	Assistant + portals	Ask PGR writes with confirm · student portal actions · notification depth

Order rationale: F1 is a governance risk (submitting a wrong return). F2 unblocks GDPR + handles the duplicate-record problem that **every** real institution hits on import. F3–F4 close the two ends of the student lifecycle (admissions in, graduation out). F5 makes operations honest. F6 is the visible-polish layer that trust hinges on.

High-impact functional gaps

[HIGH] Statutory #1. HESA return is an illustration, not a return

What the user sees today. The *Statutory returns* screen shows a HESA profile with ~12 fields, a Validate button, and a Generate button that produces a CSV.

The workaround under the hood. The 12 fields were chosen to prove the mapping engine — dotted-path resolution, transforms, per-field validation. The profile is not compliant. §13 of the impl doc admits it.

What is functionally missing. 40–70 mandatory HESA fields; coding-frame validators (e.g. SEXID, ETHNIC, DISABLE); mandatory-field enforcement at profile compile time; a sign-off record on the profile before it becomes a production year.

Fullstack build

Data model	New rows in report_profile (year 2026) + one report_field_mapping<
Service / rules	Extend exports/service.py: profile compile refuses if any mandatory HESA field is unmap
API	POST /reports/profiles/{id}/sign-off (guarded by reports.signoff</font
UI	Statutory page: show sign-off state, list of missing mandatory fields with a jump-to-mapping action, and a "can this be
Tests	Test: profile with a missing mandatory field cannot be signed off; a signed-off profile refuses further edits; generate o

[HIGH] Person #2. No merge, no contacts, no GDPR export/erase

What the user sees today. Person 360 shows identities on a timeline and looks complete.

The workaround under the hood. Duplicate person rows have no resolution path — administrators live with duplicates. Contacts do not exist; the notifier assumes one channel per user. GDPR export/erase is not implemented.

What is functionally missing. First-class `person_service.merge(surviving_id, losing_id)` that rewrites every FK, preserves audit history, keeps both original ids in the merge record. Contact model with typed channels + do-not-contact flag. Subject-access export walking every table with a person_id. Erasure that pseudonymises the person while preserving financial-audit integrity.

Fullstack build

Data model	New tables: person_merge_record (surviving_id, losing_id, merged_by, merged_at, reas
Service / rules	person_service.merge — rewrites FKs across every module module-by-module inside or
API	POST /persons/merge, GET /persons/{id}/export, <font face=
UI	Persons list: duplicate detection banner + Merge action. Person detail: Contacts card. New GDPR tab with Export an
Tests	Two duplicate persons merge to one; loser row is gone but merge record + audit trail survive; erased person cannot

[HIGH] Recruitment #3. References and interviews live in email + Word

What the user sees today. Application detail shows a stage history and assessments card. The pipeline looks complete.

The workaround under the hood. References and interviews are not domain objects. Administrators nominate referees in email, receive replies in email, then paste a summary into an assessment. Interview panels are scheduled by hand.

What is functionally missing. First-class `reference_request` with token-linked external submission form; `interview` with panellists, slot, calendar event via the outbox, and a captured outcome.

Fullstack build

Data model	New tables: reference_request (application_id, referee_email, token_hash, requested_at
Service / rules	Reference: create → token URL emitted via notifier → external submit endpoint validates token → attaches PDF/text

API	POST /applications/{id}/references, POST /public/references/{token}
UI	Application detail: References card with request/received/failed state; Interviews card with panel + outcome; referee-
Tests	Referee round-trip via token URL attaches to application; expired token refused; two panellists cannot conflict; applic

[HIGH] Admissions #4. No conditional offers, no fee-status / visa gating

What the user sees today. Offers move draft → issued → accepted / declined. Clean and simple.

The workaround under the hood. "Accept subject to your Masters at 2:1+" cannot be recorded. Fee status and visa status are not captured. International students go through unchecked.

What is functionally missing. `OfferCondition` as its own object (many per offer) with satisfy-by date and evidence reference. Fee-status + visa-required fields on applicant + offer that gate issue and acceptance.

Fullstack build

Data model	New table offer_condition (offer_id, description, satisfy_by, satisfied_at, evidence_docum
Service / rules	Offer issue refuses when visa_required and visa check is not confirmed. Offer acceptanc
API	POST /offers/{id}/conditions, POST /offers/{id}/conditions/{cid}/satisf
UI	Offer detail: Conditions card with add / satisfy / evidence upload. Applicant detail: fee-status + visa fields with a checl
Tests	Cannot issue a visa-required offer without a completed visa check; cannot accept an offer with an unsatisfied past-du

[HIGH] Completion #5. Classification is a free field; no certificate

What the user sees today. Completion screen has a Graduate button. Graduation runs — award recorded, funding closed, alumnus opened.

The workaround under the hood. Award classification (Pass / Merit / Distinction / PhD-w-corrections / PhD-w-none) is a free-text field, not an enum with a workflow. There is no PDF certificate. Records Office runs a parallel process in Word.

What is functionally missing. Classification workflow (proposed by chair → confirmed by exam board → published). Certificate PDF template with institutional typography, generated on the graduation transaction, stored in the object store, linked from the completion row.

Fullstack build

Data model	New enum Classification; add award.classification, <font face
Service / rules	Extend completion_service.graduate(): state machine on classification; on publish, rende
API	POST /completions/{id}/classify, POST /completions/{id}/publish</fo
UI	Completion detail: Classification card with propose/confirm/publish actions; Certificate card with preview and downloa
Tests	A completion cannot publish without a confirmed classification; certificate PDF opens, contains name/award/date, is

Medium-impact functional gaps

[MED] Ops #6. Workflow SLA timers do not exist

What the user sees today. The Tasks screen shows overdue counts. The worker escalates overdue rows on its 60-second interval.

The workaround under the hood. A task is either in progress or overdue against a fixed deadline. There is no notion of an institutional service level ("we promise a 5-working-day turnaround") or reporting against it.

What is functionally missing. `TaskSla` table with target duration, working-day-aware elapsed clock, breach flag. SLA report parameterised by workflow definition and window.

Fullstack build

Data model	New table task_sla (workflow_definition_id, task_type, target_seconds, working_days_or
Service / rules	Task creation: attach the applicable SLA; worker computes elapsed on a working-day calendar; breach transitions th
API	CRUD /admin/task-slas, GET /reports/sla.
UI	Settings → SLAs tab (define target per workflow type). Tasks page: SLA column with elapsed / target / breached. Re
Tests	Turnaround averages match manual calculation on a seeded dataset; a task past its target moves to breached within

[MED] Integration #7. Reconciliation UI shows problems but cannot remediate

What the user sees today. The Integration hub renders three sections: stuck outbound, failed inbound, waiting on a person.

The workaround under the hood. Detection only. There is no bulk retry, no inline resolve, no export. Dead-letter replays are one-at-a-time.

What is functionally missing. Bulk actions on the three lists: replay N dead-letters in one action; resolve waiting-on-a-person by picking the person inline; export the reconciliation queue to CSV.

Fullstack build

Data model	No schema change.
Service / rules	Extend integration/service.py with replay_many(ids), <font fa
API	POST /integration/dead-letters/replay (body: ids), POST /integration
UI	Reconciliation panel: multi-select checkboxes + Replay selected; inline person picker in the waiting list; E
Tests	Selecting 20 dead-letters and replaying leaves 20 audit rows tagged with the actor and outcome; resolving a waiting-

[MED] Funding #8. Old students break the integrity screen

What the user sees today. The Funding integrity screen highlights every student whose funding chain has a problem, errors first.

The workaround under the hood. Pre-Phase-6 students have no `research_project` row and therefore no link to their award — so the integrity check screams at every one of them. §13 admits it and there is no backfill.

What is functionally missing. One-shot backfill: for each pre-Phase-6 student, either link the existing project (best-guess by department + award) or create a stub project linking the student to the funding source of record. Log every uncertain case for manual review.

Fullstack build

Data model	No schema change. Uses existing research_project.
Service / rules	backfill_project_links.py — dry-run mode by default; produces a report of sure-linked / st

API	N/A (script + a Settings toggle to acknowledge that backfill has been run).
UI	Settings → Data health card: shows last-backfill timestamp + counts; a link to the uncertain-cases task list.
Tests	After running on a prod-copy, funding-integrity errors drop to zero (or every remaining error is a real one). Uncertain

[MED] Assistant #9. Read-only + unresolved Tier-3 write policy

What the user sees today. Ask PGR answers natural-language questions accurately. Users assume they can also ask it to *do* things.

The workaround under the hood. By design, this release is read-only. The Tier-3 blocked-action list has one unresolved item. UI shows "Read-only — I can't change anything".

What is functionally missing. 5.2 writes with a mandatory confirm-before-write step per the pattern in `docs/PGR_ASSISTANT_DESIGN.md`. Tier-3 policy signed off first.

Fullstack build

Data model	New table assistant_write_intent (intent_json, preview_json, actor, confirmed_at, execute
Service / rules	Extend assistant/tools with propose_write tools that return a preview + intent id.
API	POST /assistant/intents (create intent, return preview), POST /assis
UI	Assistant palette: after a write intent, render a diff panel with Confirm / Cancel. Every executed intent lands in the aud
Tests	Writes require a confirm click; Tier-3 blocked actions are refused with the exact rule cited; audit trail names the assis

[MED] Portals #10. Student portal actions are cosmetic

What the user sees today. My journey renders milestones, meetings, funding, thesis, tasks. Feels rich.

The workaround under the hood. Almost read-only — a couple of upload buttons and nothing else. Students still email the office to accept meeting slots, upload corrections, or acknowledge conditions on a progression outcome.

What is functionally missing. Two real flows first: (a) student uploads correction evidence → workflow triggers supervisor sign-off → admin completes; (b) supervisor proposes N meeting slots → student picks one → meeting created with the correct people and date.

Fullstack build

Data model	New tables: meeting_proposal (supervisor_id, student_id, slots_json, status); <font face=
Service / rules	Corrections flow: submit → notification to supervisor → sign-off → notification to admin → complete. Meeting flow: pr
API	POST /portal/corrections, POST /portal/meetings/propose, <
UI	Portal: Corrections card with upload + status; Meetings card with pending proposals + accept action. Supervisor port
Tests	Student uploads a correction → supervisor sees it in their queue → signs off → admin closes; every step audited. Me

[MED] Notifications #11. Delivery works; hygiene does not

What the user sees today. Notifications appear in the notification centre and are emailed to users who opted in.

The workaround under the hood. Templates are minimal and unbranded. No quiet hours, no digest, no unsubscribe. Bounces are silent — the platform keeps mailing an address that has been dead for months.

What is functionally missing. Branded HTML templates. Per-user quiet hours + daily-digest preference. List-unsubscribe header. Bounce handler that deactivates the email channel automatically.

Fullstack build

Data model	Add notification_preference.quiet_start, quiet_end, <font face
Service / rules	Notifier: check quiet hours before send; digest job at 08:00 collapses queued notifications into one email. Bounce we
API	PUT /notifications/preferences (extended), <font face='Courier'>POST /webhooks/email/

UI	Settings → My preferences: quiet hours picker + digest toggle. Person detail: bounce badge if the email is bad.
Tests	A hard bounce deactivates the channel on the next tick; quiet-hours delays a non-urgent notification and it arrives aft

Lower-impact polish (single-table)

#	Area	Gap	Fullstack change
12	Recruitment	No route badge / filter on pipeline	UI-only: add column + chip filter on /recruitment/applications; server already
13	Reporting	No cohort or trend reports; forecast has no backtest	New reporting endpoints parameterised by intake year; add confidence banc
14	Supervision	No supervisor leave / delegation record	New supervisor_leave table; capacity guard skips supervisors on leave; dele
15	Milestones	Milestone definitions are code-seeded, not visibly editable	Settings → Programme milestones tab: CRUD milestone_definition per prog
16	Thesis	Corrections have no per-item checklist	New thesis_correction_item table; UI list with tick-off; existing sign-off gates
17	Documents	No versioning or retention on uploads	Add document_version table; on re-upload keep prior version; lifecycle rule c
18	Audit	No search by field-level change (only actor/action/entity)	Store JSON diff on mutating rows; add /audit/search by diff key/value; UI diff
19	Analytics	Risk reasons rendered as tags — no drill-through	UI-only: clicking a rule tag shows the qualifying students for that rule.
20	Persons	No relationship types beyond generic pair	Extend person_relationship.kind enum (spouse, parent, guardian, emergenc
21	Recruitment	Assessment scoring is free-text	New assessment_rubric per opportunity with weighted criteria; existing asse
22	Global	No global saved views / filters	New saved_view table (per-user, per-list); UI "save this view" on Students / A

Cross-cutting functional discipline

Threads that run through every gap above and should be enforced **once** so we don't reinvent the pattern per feature.

Discipline	Enforce by
Every new mutating endpoint is guarded and audited	Route decorator + AuditMiddleware — already the standard; add lint check.
Every new list endpoint carries page + total	Response schema base class already exists — reuse, don't re-implement.
Every new schema field is added to create + read schemas	Pydantic silently drops missing create fields — add test that diffs create vs. model.
Every workflow state has a task + notification + outbox event	workflow_definition seed migrations; test asserts each state has its trio.
Every new domain enum has a migration + seed + LOV + picker	One PR pattern: enum → migration → LOV → picker component.
Row-scoping applied on any new list	Add repository fixture that runs list endpoints as a supervisor and asserts scope.

Definition of done for a functional gap. The screen reflects the underlying model, not a shortcut. There is a test that exercises the flow end-to-end. The audit trail names the actor. The workaround comment in code is deleted, not preserved.